

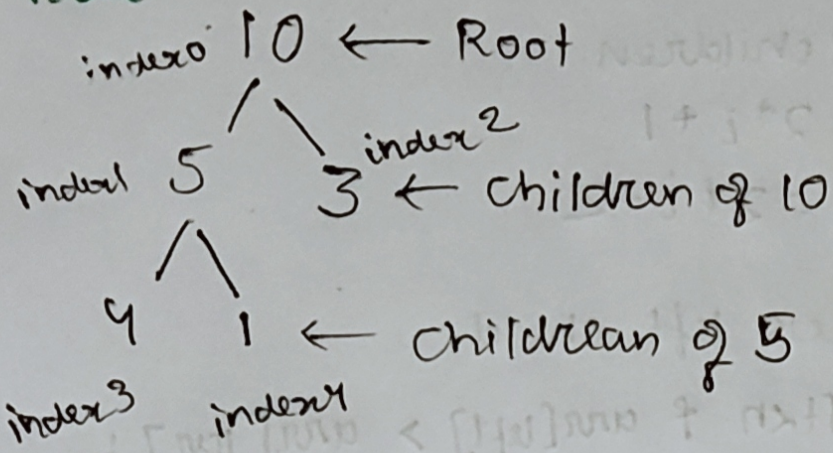
Heap Sort

Array

arr = [4, 10, 3, 5, 1]

index 0 1 2 3 4

tree



math

left-child = $2 * i + 1$

right-child = $2 * i + 2$

Ex - $i = 0$

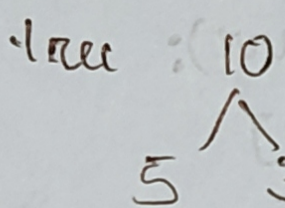
left = $2 * 0 + 1 = 1$

right = $2 * 0 + 2 = 2$

so arr[0] = 10

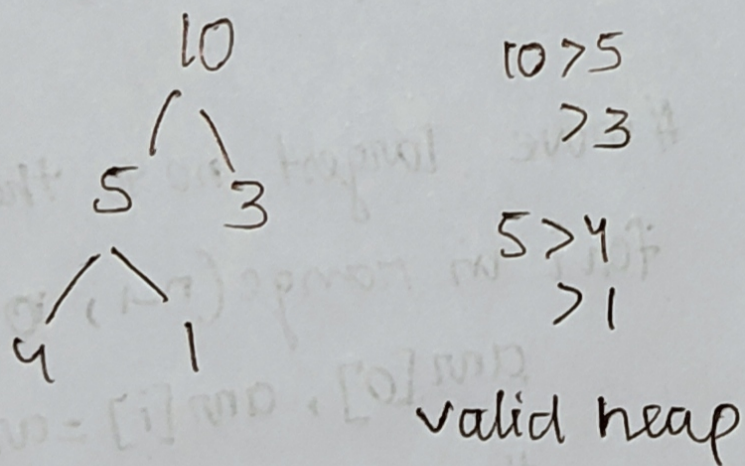
left child arr[1] = 5

right child arr[2] = 3



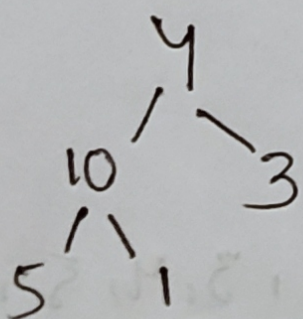
valid

Heap Sort normally used
 max heap



The biggest no.

is always at the top



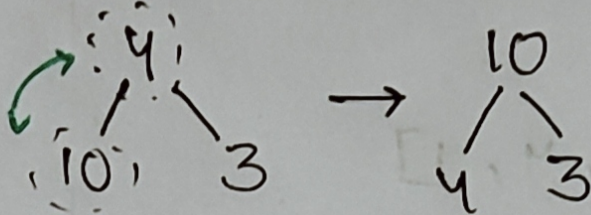
arr = [4, 10, 3, 5, 1]

not valid max heap $4 > 10 \times [4, 10, 3, 5, 1] \rightarrow [10, 4, 3, 5, 1]$

Heapify

heapify()

fix this part so the tree so that the biggest value becomes the parent



def heapify(arr, n, i):

largest = i

left = $2 * i + 1$

right = $2 * i + 2$

if left < n and arr[left] > arr[largest]:
 largest = left

if right < n and arr[right] > arr[largest]:
 largest = right

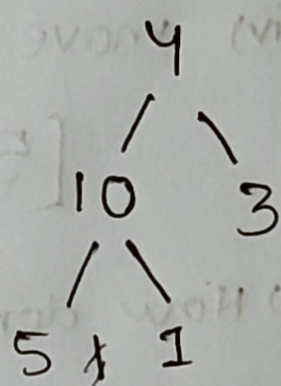
if largest != i:

arr[i], arr[largest] = arr[largest], arr[i]

heapify(arr, n, largest)

Ex -

arr = [4, 10, 3, 5, 1]



(i) first $i = 0$

largest = 4

then left = $2 * i + 1$

= 1

arr[1] = 10

right = $2 * i + 2$

= 2

arr[2] = 3

(ii) arr[left] > arr[largest]

$10 > 4 \rightarrow \text{yes!}$

largest = left = 10

(iii)

is $3 > 10$?

No

then no change

(iv) swap

arr[i], arr[largest] = arr[largest], arr[i]

4 10

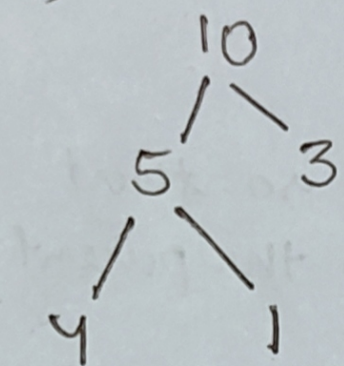
10

10

4

$4 > 10 \times [4, 10, 3, 5, 1] \rightarrow [10, 4, 3, 5, 1]$

Heap Sort



arr [10, 5, 3, 4, 1]

but we want [1, 3, 4, 5, 10]

(ii) Swap 1st & last element

[10, 5, 3, 4, 1] → [1, 5, 3, 4, 10]

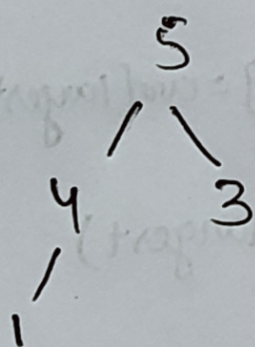
swap 10 & 1

(iii) Don't touch 10 → [1, 5, 3, 4]

This is no more
max heap

↓ heapify()

[5, 4, 3, 1, 10]



(iv) move 5 to end

[5, 4, 3, 1] → [1, 4, 3, 5, 10]

(v) Now don't touch 5 & 10. [1, 4, 3]

↓ heapify()

[1, 3, 4]

swap

[4, 3, 1]

(vi) Eventually

[1, 3, 4, 5, 10]

Full code

```
def heapify(arr, n, i):
```

```
# assume the current node is largest
largest = i
```

```
# Find children
```

```
left = 2*i + 1
```

```
right = 2*i + 2
```

```
# check left child
```

```
if left < n and arr[left] > arr[largest]:
```

```
largest = left
```

```
# check right child
```

```
if right < n and arr[right] > arr[largest]:
```

```
largest = right
```

```
# if one of children is bigger
```

```
if largest != i:
```

```
# swap parent w/ biggest child
```

```
arr[i], arr[largest] = arr[largest], arr[i]
```

```
def heap_sort(arr):
```

```
n = len(arr)
```

```
# Build max heap
```

```
for i in range(n//2 - 1, -1, -1):
```

```
heapify(arr, n, i)
```

```
# move largest no to the end
```

```
for i in range(n - 1, 0, -1):
```

```
arr[0], arr[i] = arr[i], arr[0]
```

```
# restore max heap
```

```
heapify(arr, i, 0)
```

```
arr = [4, 10, 3, 5, 1]
```

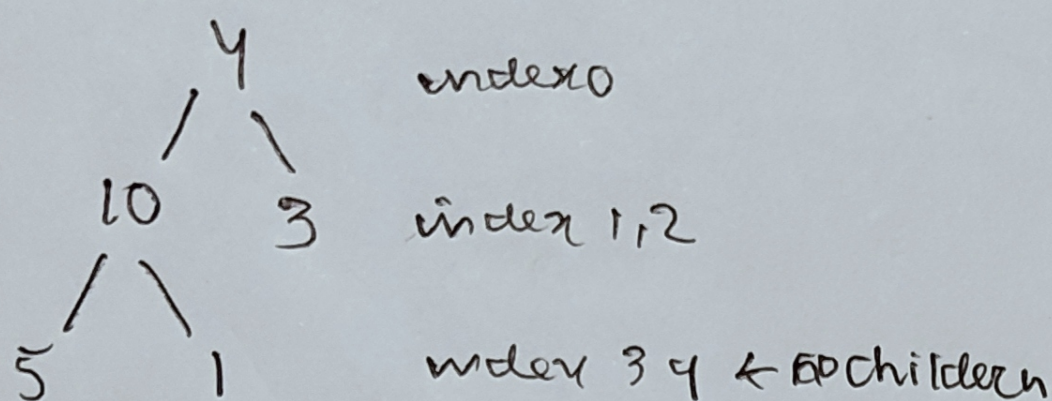
```
heap_sort(arr)
```

```
print(arr) # [1, 3, 4, 5, 10]
```

code short

$(n//2 - 1, -1, -1) \rightarrow ?$

because roughly the 2nd half of the array contain nodes w/ no children



$\text{range}(\text{start}, \text{stop}, \text{step})$

doesn't include

$\text{range}(5, 0, -1)$

$\text{range}(4, -1, -1)$

5
4
3
2
1

4
3
2
1
0

arr = [4, 10, 3, 5, 1]

$n = 5$

$n//2 = 5//2 = 2$

$n//2 - 1 = 2 - 1 = 1$

$\text{range}(n//2, -1, -1)$

$(1, -1, -1)$

$\{ 0 \}$

max heap $\rightarrow [10, 5, 4, 3, 1]$

for i in $\text{range}(4, 0, -1)$

$i = 4$

$\text{arr}[0], \text{arr}[4] = \text{arr}[4], \text{arr}[0]$

Before $[10, 5, 4, 3, 1]$

after

$[1, 5, 4, 3, 10]$

0

1

2

3

4

0

1

2

3

4

$\text{arr}[0]$

$\text{arr}[4]$

$\text{arr}[4]$

$\text{arr}[0]$

$\text{arr}[0]$